

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No. 3 (A)

Aim : Using JFLAP, create a DFA from a given regular expression.

Objective: Simulate a program to implement even number of Zero's in String

- a. Using Jflap tool
- b. Using finite automata

Outcome: Students are able implement even number of Zero's in String.

Theory:

What is JFLAP: -

JFLAP program makes it possible to create and simulate automata. Learning about automata with pen and paper can be difficult, time consuming and error-prone. With JFLAP we can create automata of different types and it is easy to change them as we want. JFLAP supports creation of DFA and NFA, Regular Expressions, PDA, Turing Machines, Grammars and more.

Setup: -

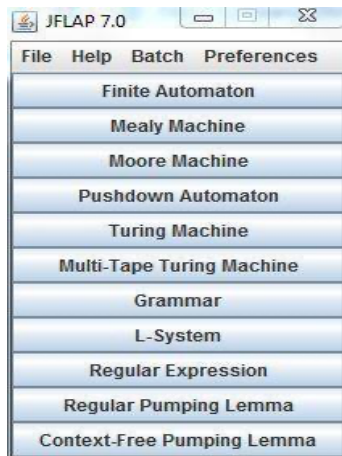
JFLAP is available from the homepage: (www.JFLAP.org). From there press "Get FLAP" and follow the instructions. You will notice that JFLAP have a .JAR extension. This means that you need Java to run JFLAP. With Java correctly installed you can simply select the program to run it. You can also use a command console run it from the files current directory with, Java -jar JFLAP.jar.

Using JFLAP: -

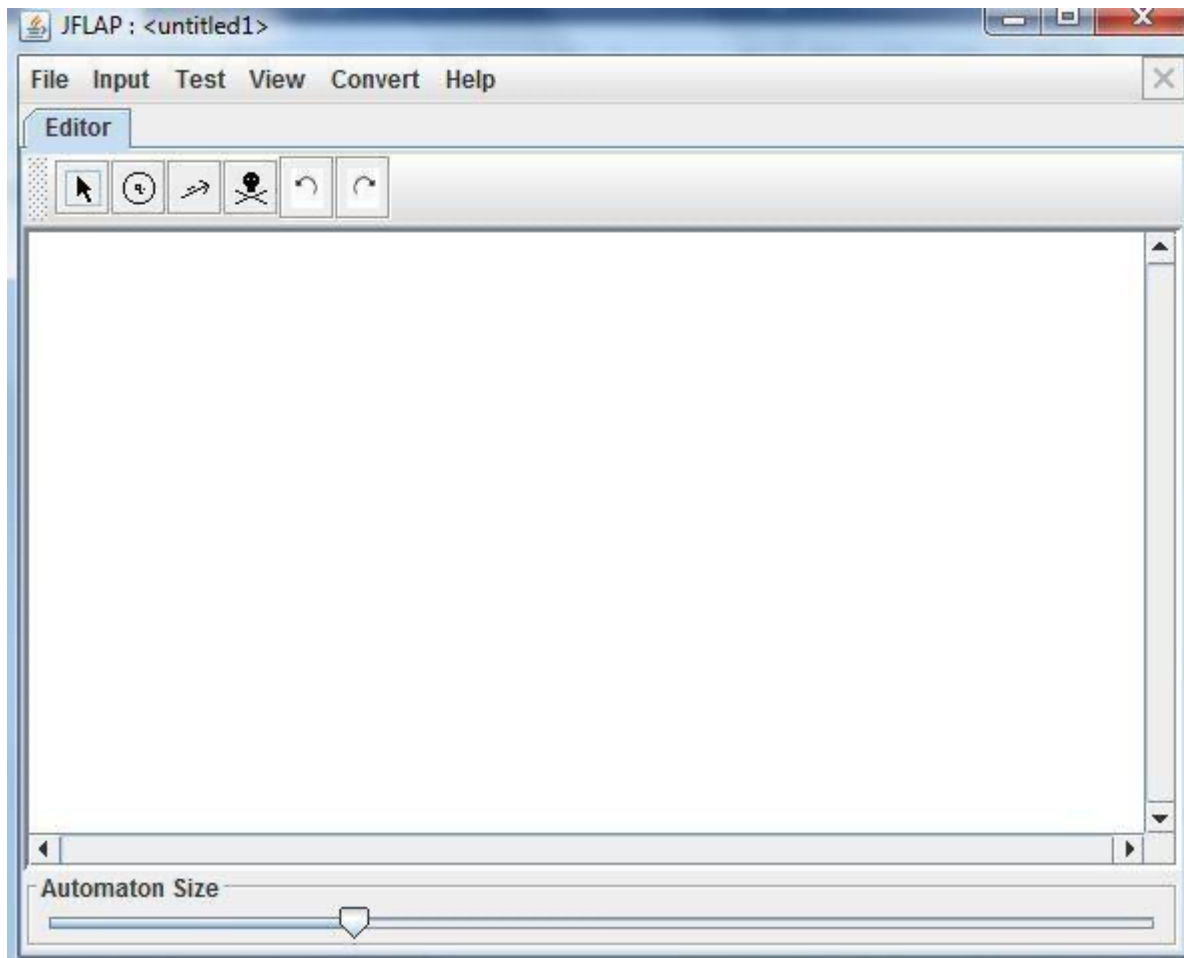
When you first start JFLAP you will see a small menu with a selection of eleven different automata and rule sets. Choosing one of them will open the editor where you create chosen type of automata. Usually you can create automata containing states and transitions but there is also creation of Grammar and Regular Expression which is made with a text editor.

DFA from a given regular expression: -

First, we need to select Regular Expression from the JFLAP Menu.



Now you should have an empty window in front of you. You will have a couple of tools and features at your disposal.



The toolbar contains six tools, which are used to edit automata.

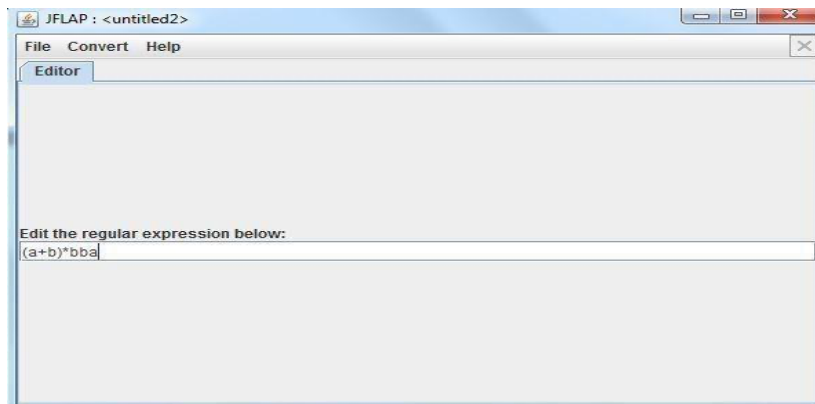
Attribute Editor Tool, changes properties and position of existing states and transitions.

State Creator Tool, creates new states.

Transition Creator Tool, creates transitions. Deletion Tool, deletes states and transitions.

Undo/Redo, changes the selected object prior to their history.

Regular Expressions can be typed into JFLAP to be converted to an NFA

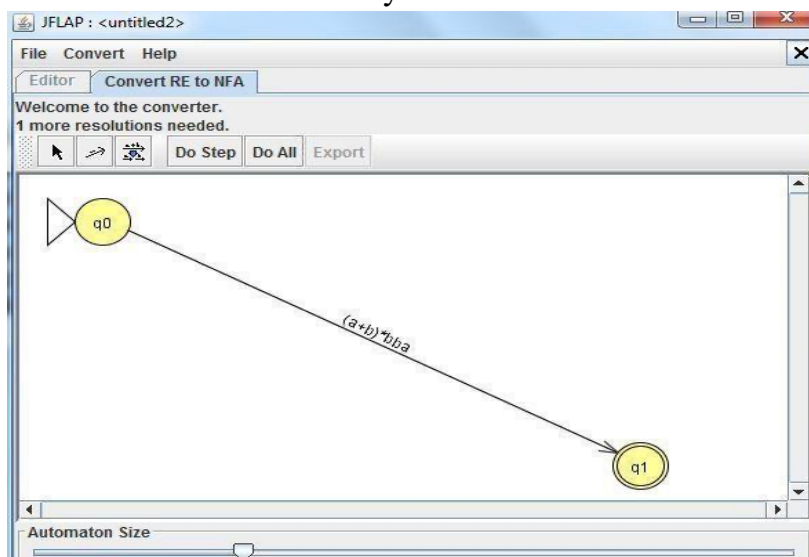


Choose Regular Expression in the main menu, then just type the expression in the textbox. Definitions for Regular Expressions in JFLAP:

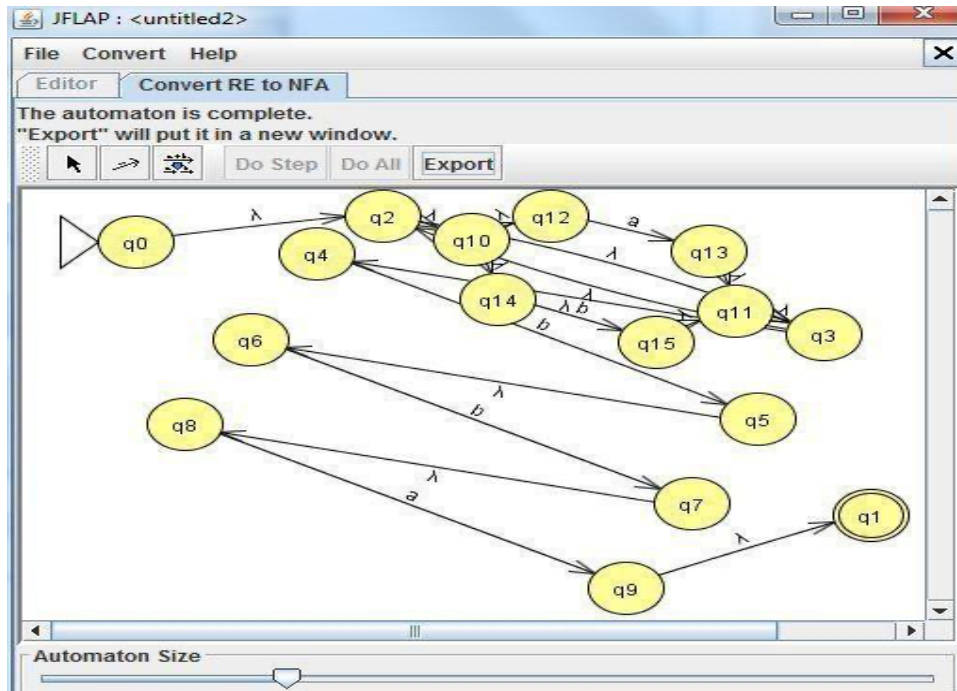
- Kleene Star
- ☐ + Union
- ! Empty String

Correctly written expressions can then be converted to an NFA. To convert your expression select Convert→Convert to NFA. The conversion will begin with two states and a transition with your RegularExpression. With the (D)e-expressionify Transition tool you can break down the Regular

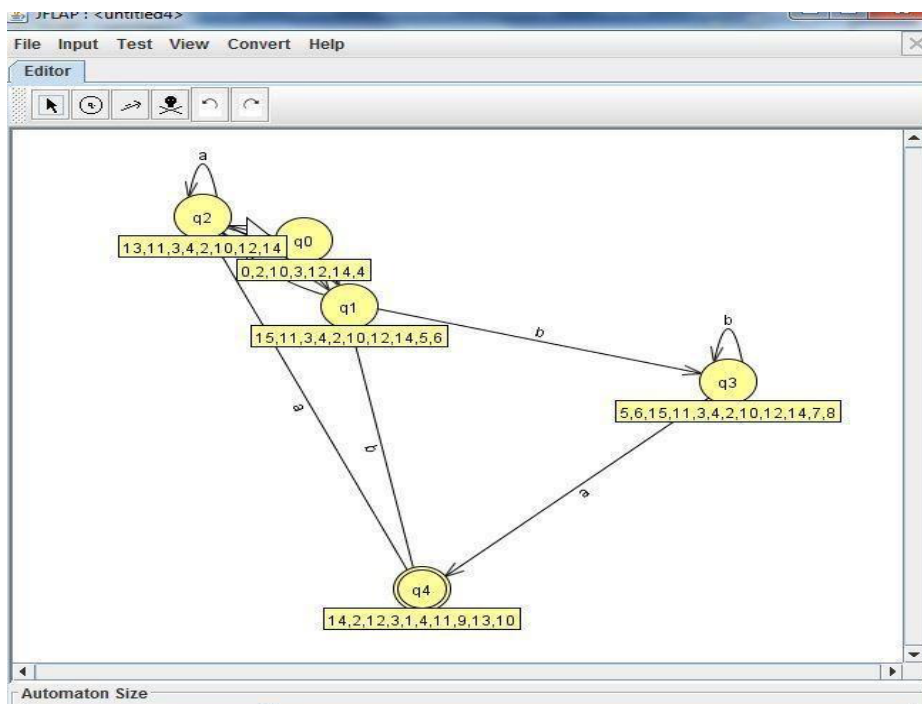
Expression into smaller parts. Each transition will contain a sub expression. The next step is to link every rule with lambda transitions. Add new transition between states that should be connected with the Transition Tool. If you are unsure what to do you can select Do Step to automatically make the next step. If you want the NFA immediately Do All creates the whole NFA for you.



You can notice how the conversion differs depending on how the Regular Expression looks. For example the expression $a+b$ results in a fork, where either 'a' or 'b' can be chosen.



Now finally convert NFA to DFA:-



PART B

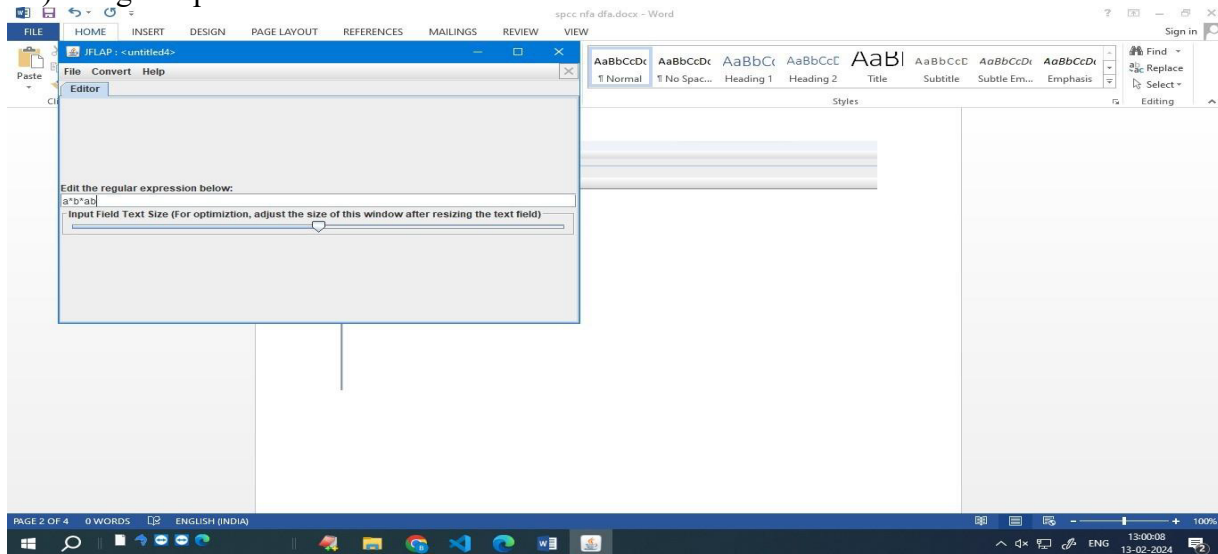
(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

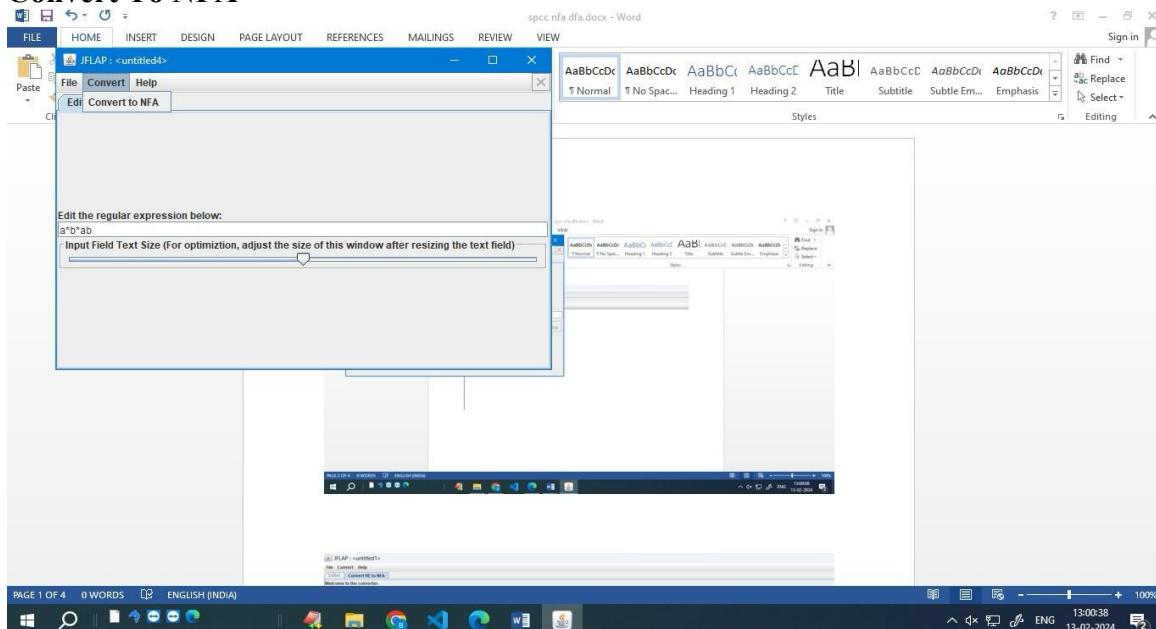
Roll. No. C-54	Name: Quazi Md Moinuddin
Class: TE-C	Batch: C-3
Date of Experiment: 25.03.2025	Date of Submission: 07S.04.2025
Grade:	

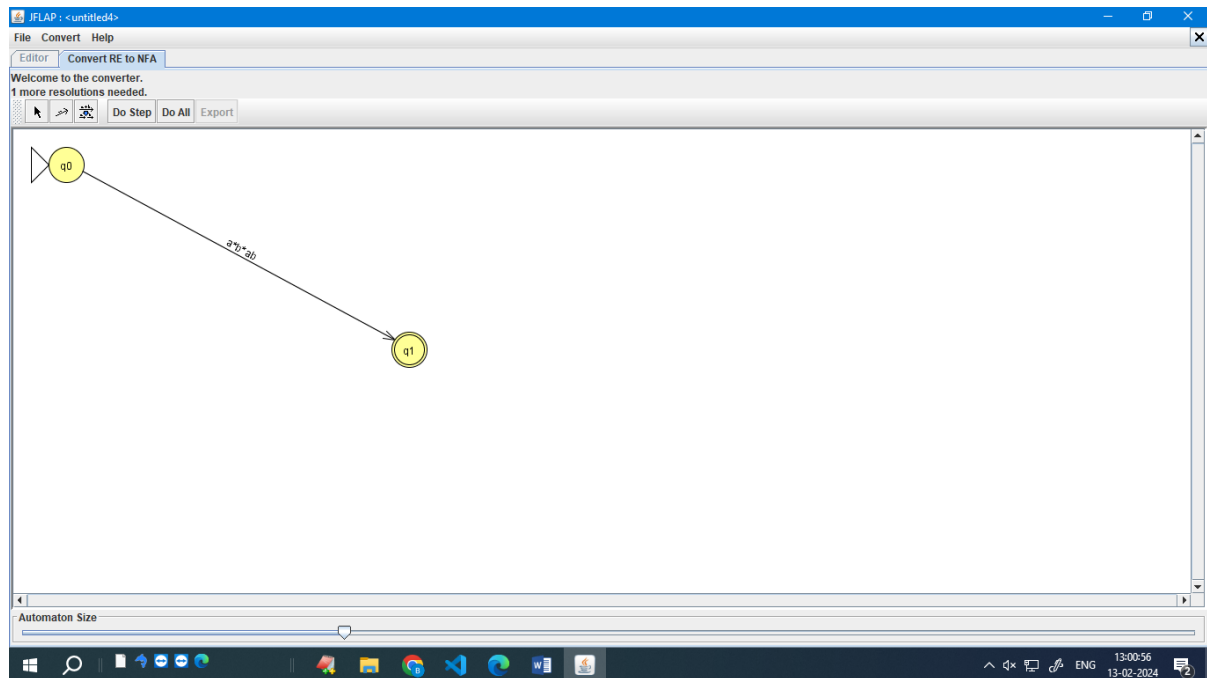
B.2 Input and Output: Regular Expression

a) Using JFlap tool

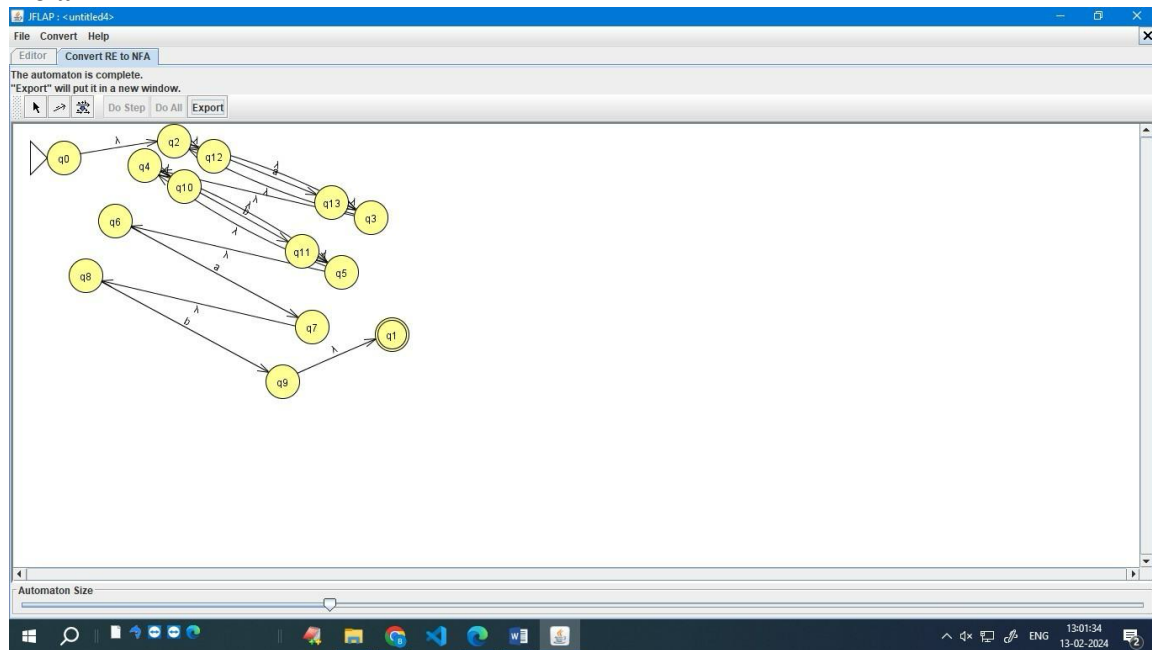


Convert To NFA

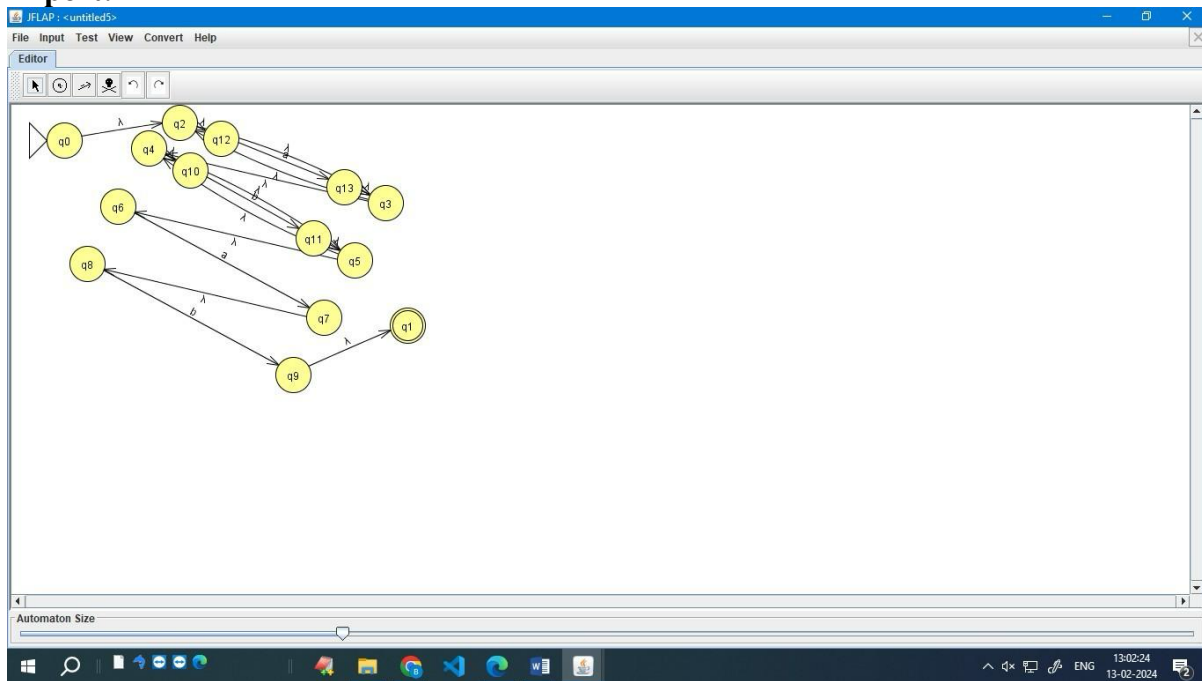




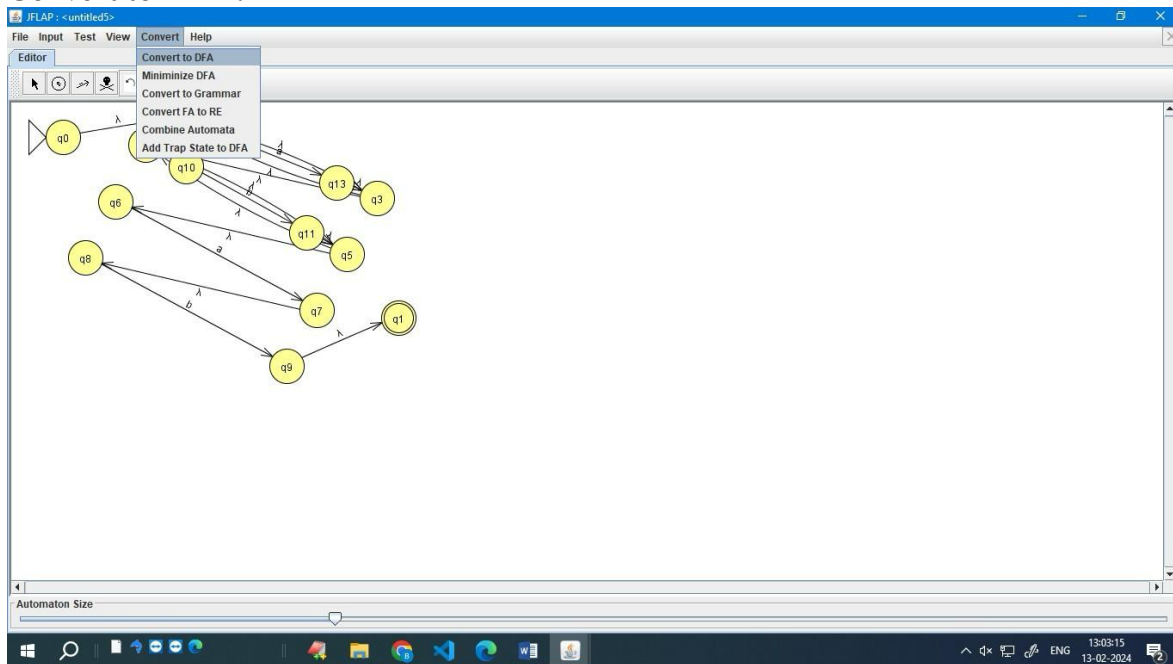
Do all

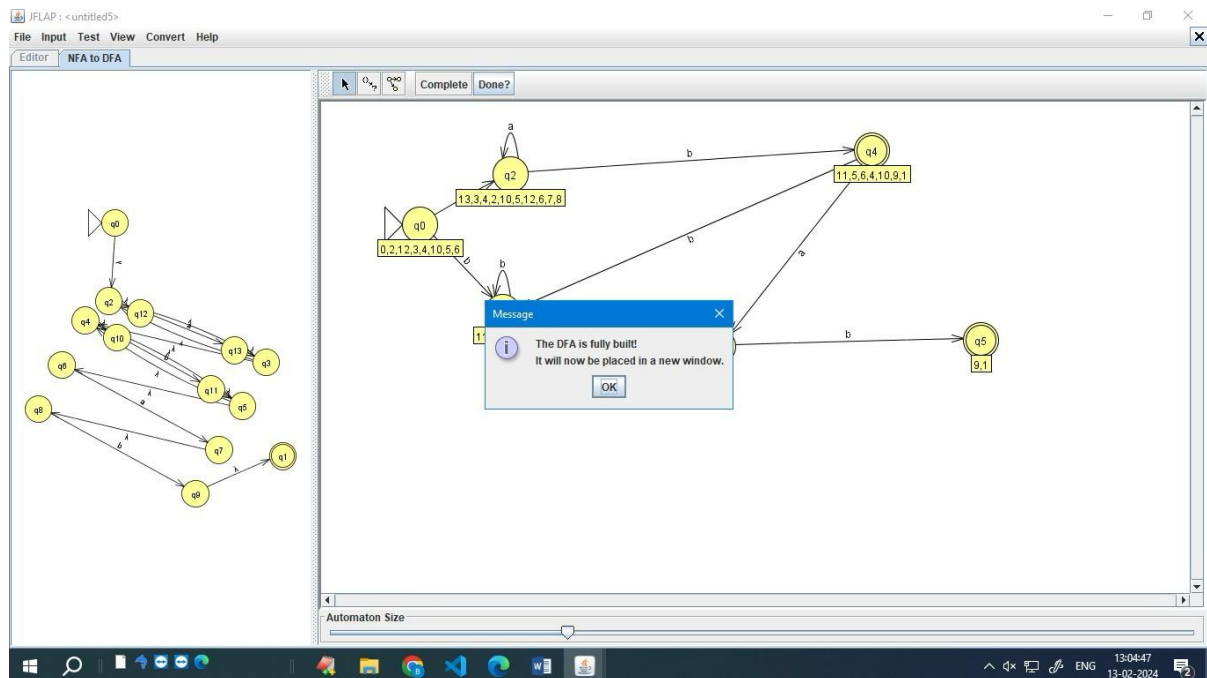
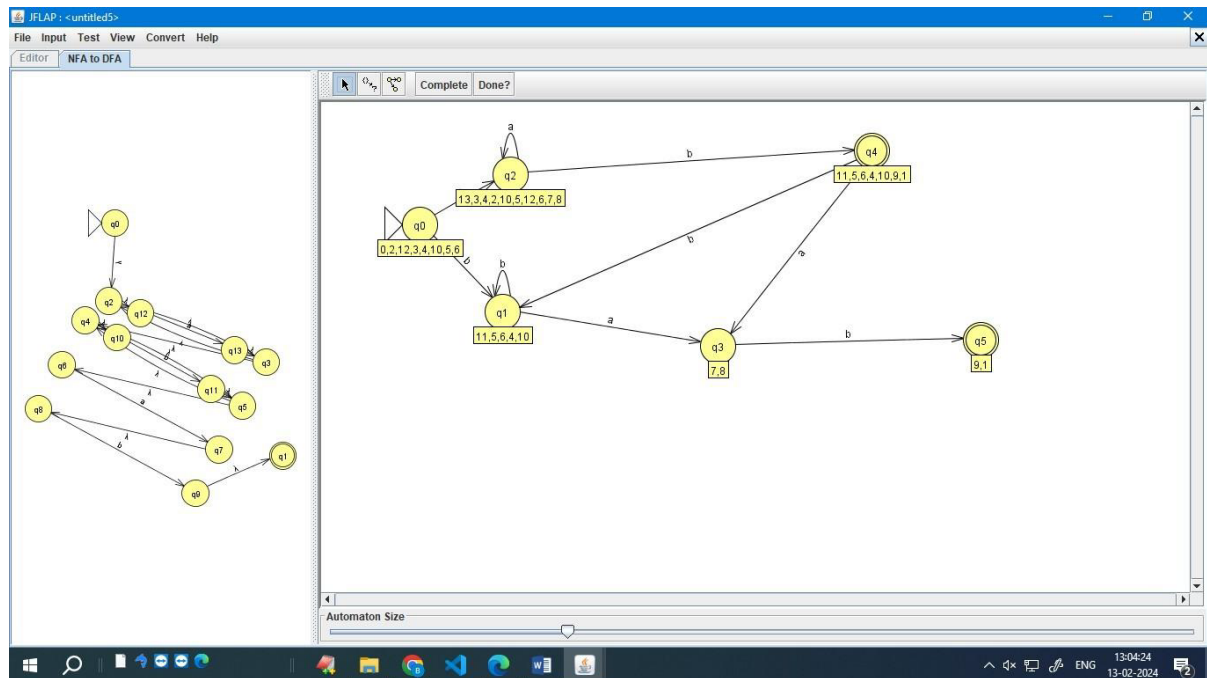


Export:



Convert to DFA :





b) using programm

B.3 Observations and learning:

JFLAP's user-friendly interface streamlines DFA creation from a regular expression, highlighting the stepwise NFA conversion and the utility of tools like (D)e-expressionify Transition and learned to emphasizes the interplay between regular expressions and automata design, enhancing comprehension of NFA construction stages and the role of specialized tools in simplifying complex expressions during refinement.

B.4 Conclusion:

The experiment demonstrated successful creation of a DFA from a regular expression using JFLAP. The process involved menu selection, input typing, NFA conversion, and refinement with tools like (D)e-expressionify Transition. JFLAP's visual representation enhanced understanding, highlighting its role in simplifying automata design and reinforcing theoretical concepts.

B.5 Question of Curiosity

(To be answered by student based on the practical performed and learning/observations)

1. How does JFLAP tool help in designing finite automata?

Ans : JFLAP simplifies the design process of finite automata by providing a user-friendly interface to create and simulate different types of automata. Its tools, such as the State Creator Tool and Transition Creator Tool, streamline the creation of states and transitions, making it easier to visualize and understand automaton structures.

2. How JFLAP can be used to design programming language?

Ans : JFLAP aids in designing programming languages by allowing the creation and simulation of various automata, including DFAs, NFAs, and regular expressions. This capability is crucial in language design, where automata play a role in lexical analysis and pattern recognition, forming the foundation for constructing language syntax and semantics.

3. Can we convert every DFA to NFA?

Ans : Yes, every Deterministic Finite Automaton (DFA) can be converted to a Non-deterministic Finite Automaton (NFA). The conversion involves creating equivalent states and transitions to capture the deterministic behavior of the original DFA. This process is supported by tools like JFLAP, which can automatically perform the conversion, aiding in the exploration of different automaton representations.